

Retrieval Engine Competition Project

Cleaned structure, study workbook, and report

Project Goal

Build a retrieval pipeline for the Kaggle competition, keep the code runnable, and turn the repository into a better study object by consolidating scattered notebooks into one workbook and one A4 report.

Project	Retrieval engine competition pipeline
Main code	<code>src/ + main.py</code>
Workbook	<code>notebooks/retrieval_project_workbook.ipynb</code>
Report	<code>report/retrieval_project_report.pdf</code>
Date	March 11, 2026

Contents

1	Project At A Glance	2
1.1	Clean Repository Layout	2
2	Dataset Snapshot	2
3	Steps We Took	3
4	Modeling Choices	3
4.1	TF-IDF	3
4.2	BM25+	3
4.3	Hybrid Semantic Reranking	4
4.4	Fast Study Benchmark	4
5	Submission Workflow	4
6	How To Study The Project	4
A	Appendix: Complete Explanations	5
A.1	Why The Shared <code>content</code> Field Matters	5
A.2	Metric Formulas	5
A.3	Code Walkthrough	5
A.4	Practices And Active Learning Exercises	6
A.5	Practical Study Advice	6

1 Project At A Glance

The cleaned repository now separates execution, learning, and reporting:

- `src/` contains the runnable retrieval system.
- `notebooks/retrieval_project_workbook.ipynb` is the single teaching-first notebook.
- `report/retrieval_project_report.pdf` is the formal A4 summary with appendix material.

What Was Removed

The old notebook collection had overlapping responsibilities: data exploration, explanation, Kaggle submission, model intuition, and reporting were split across several files. Those were merged into one workbook so the learning path is linear instead of fragmented.

1.1 Clean Repository Layout

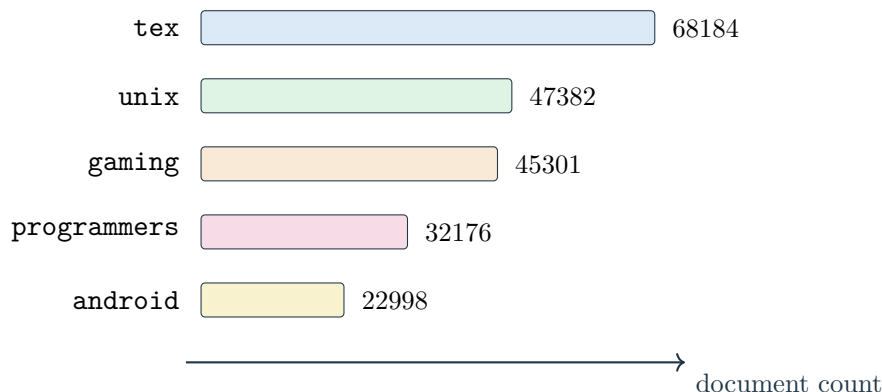
Path	Purpose
<code>src/config.py</code>	Central runtime settings for evaluation and submission generation.
<code>src/preprocess.py</code>	Builds the shared <code>content</code> field for documents and queries.
<code>src/models.py</code>	TF-IDF, BM25+, dense retrieval, and hybrid reranking.
<code>src/evaluation.py</code>	Precision@K, Recall@K, MRR, and MAP.
<code>src/utils.py</code>	Data loading, CSV writing, and submission validation.
<code>src/pipeline.py</code>	End-to-end orchestration with clearer helper functions.
<code>notebooks/retrieval_project_workbook.ipynb</code>	One notebook with plots, diagrams, exercises, and code explanations.
<code>report/retrieval_project_report.pdf</code>	Final source for this A4 report.

2 Dataset Snapshot

The local data used in this project contains:

- 216,041 documents,
- 327 training queries,
- 141 test queries,
- 327 qrels entries for evaluation.

The document categories are strongly imbalanced, which matters because retrieval behavior often changes by domain vocabulary.



Small But Important Observation

The dataset contains one empty document body. This is a good reminder that retrieval systems are not only ranking systems; they are also data-cleaning systems.

3 Steps We Took

1. **Inspected the raw JSON files.** We identified the shared fields used across documents and queries: title, text, tags, and category.
2. **Normalized the text path.** We created one common `content` field so all models compare like with like.
3. **Implemented multiple retrieval strategies.** The project supports TF-IDF, BM25+, dense retrieval, and a lexical-plus-semantic hybrid.
4. **Evaluated with ranked-retrieval metrics.** We measured Precision@K, Recall@K, MRR, and MAP on the training split.
5. **Generated Kaggle-ready files.** The pipeline writes per-model CSV files and validates the final submission against the sample template.
6. **Cleaned the repository for learning.** We replaced the notebook sprawl with one workbook and one report.



4 Modeling Choices

4.1 TF-IDF

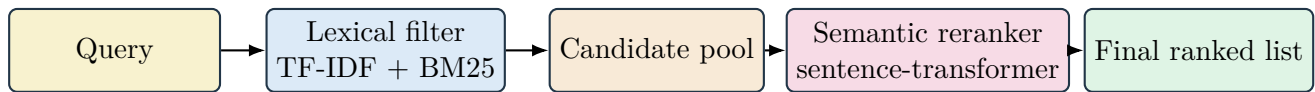
TF-IDF is a strong lexical baseline because it rewards terms that are frequent in a document but not frequent everywhere in the corpus. In this project, it uses unigram and bigram features and cosine similarity.

4.2 BM25+

BM25+ is another lexical ranker, but its scoring curve differs from TF-IDF. It usually handles term frequency saturation and document length normalization more explicitly, which can change the top-ranked items even when both models use the same input text.

4.3 Hybrid Semantic Reranking

The hybrid model works in two layers:



This is a useful design because a cheap lexical stage keeps recall high, while the semantic stage can repair some vocabulary mismatch errors.

Memorable Mental Model

Think of the hybrid system as a two-stage interview process. TF-IDF and BM25 decide who gets invited into the room. The semantic reranker decides who should sit at the front of the table.

4.4 Fast Study Benchmark

Inside the workbook, we run a smaller experiment on a `tex` subset with 30,000 documents and 12 queries so the student can iterate quickly.

Model	Precision@10	Recall@10	MRR@10	MAP@10
TF-IDF	0.0750	0.3639	0.1605	0.1159
BM25+	0.0500	0.2042	0.3417	0.1646

These values are not presented as the final competition result. They are a teaching benchmark designed to make metric tradeoffs visible quickly.

5 Submission Workflow

The project generates:

- one CSV per configured model,
- one final upload file based on the chosen `final_model`,
- a validation check against the sample submission template.

Centralizing runtime settings in `src/config.py` makes the pipeline easier to explain and safer to modify. Instead of hunting through `src/pipeline.py` for scattered constants, the user now has one place to adjust the main switches.

6 How To Study The Project

Recommended order:

1. Open the workbook and run the data summary cell.
2. Study the category and length plots before reading the models.
3. Run the pipeline diagram cell and explain it aloud from memory.

4. Use the toy example to predict the ranking before executing the code.
5. Run the fast benchmark and compare TF-IDF against BM25+.
6. Read the appendix and complete at least two active-learning exercises.

A Appendix: Complete Explanations

A.1 Why The Shared `content` Field Matters

The data files contain several text sources. Documents have title, body text, and tags. Queries have title and text. If different models read different text combinations, then model comparisons stop being fair. The project avoids that problem by building one normalized `content` field for every item. Lowercasing is also done centrally so lexical models behave consistently.

A.2 Metric Formulas

$$\text{Precision@K} = \frac{\text{relevant items in top K}}{K}$$

$$\text{Recall@K} = \frac{\text{relevant items in top K}}{\text{all relevant items}}$$

$$\text{MRR@K} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\text{rank of first relevant result for } q}$$

$$\text{AP@K} = \frac{1}{|R_q|} \sum_{k=1}^K \text{Precision@k} \cdot \mathbf{1}[\text{item at } k \text{ is relevant}]$$

MAP is the mean of AP over queries.

A.3 Code Walkthrough

File	Explanation
<code>src/config.py</code>	Stores evaluation and submission settings in one dataclass so the pipeline is easier to configure and explain.
<code>src/preprocess.py</code>	Converts mixed cell values to text, joins selected fields, lowercases them, and guarantees a shared <code>content</code> representation.
<code>src/models.py</code>	Implements four retrieval paths. TF-IDF uses sparse lexical vectors, BM25+ uses token-based probabilistic ranking, dense retrieval encodes text into embeddings, and the hybrid path reranks lexical candidates semantically.
<code>src/evaluation.py</code>	Calculates retrieval metrics query by query, skipping queries without ground-truth relevance labels.
<code>src/utils.py</code>	Reads JSON data, converts qrels into a lookup map, writes Kaggle CSV files in the correct row order, and validates submission format.
<code>src/pipeline.py</code>	Splits the full workflow into preparation, evaluation, and submission-generation helpers so the code reads as a sequence of project steps instead of one large function.

File	Explanation
<code>main.py</code>	Keeps the command-line entrypoint minimal by delegating work to the pipeline module.

A.4 Practices And Active Learning Exercises

Exercise 1: Preprocessing Sensitivity

Remove `tags` from the document `content` field and rerun the workbook benchmark. Write down which queries changed most and why.

Exercise 2: Metric Tradeoffs

Change `top_k` from 10 to 5. Predict which metric should increase and which one may decrease before you run the code.

Exercise 3: Domain Shift

Repeat the fast study benchmark in another category such as `android` or `unix`. Compare whether the same model still wins on the same metric.

Exercise 4: Error Analysis

Pick one query where TF-IDF and BM25+ disagree. Inspect the titles of the top five retrieved documents and classify the error: vocabulary mismatch, noisy text, missing context, or data issue.

Exercise 5: Hybrid Reasoning

Without running the hybrid model, identify two queries that would probably benefit from semantic reranking. Explain what lexical retrieval is likely to miss.

Exercise 6: Code Reading

Read `run_tfidf_search`, `run_bm25_search`, and `evaluate_retrieval`. Then explain the pipeline aloud in five sentences without opening the files again.

A.5 Practical Study Advice

- Always connect a metric change to a ranking change on at least one real query.
- Use the pipeline diagram until you can redraw it from memory.
- Treat the workbook as the interactive lab and this PDF as the compact revision sheet.
- Do not jump to the hybrid model before you can explain lexical retrieval clearly.